

DETAILED REPORT

on

Quadruped Robotic Dog (PHASE 2)

by

Praveen Rengarajan (20244512), Vikrant Kumar Singh
(20244174), Pratyaksh Tiwari (20246114), Suyash Shukla
(20246149), Sourabh Choyal(20246143), Himanshu Yadav
(20246068)

(B.Tech IV Semester)

A report submitted in order to fulfil the requirements for completing the

Robotics Club Summer Project 2026



Motilal Nehru National Institute of Technology Allahabad
Prayagraj, UP, India, 211004

May 18, 2026

DECLARATION

We, as part of the 'Robotics Club Summer Project 2025' declare that this Colloquium report titled "Quadruped Robotic Dog" is wholly our own work and has not been submitted anywhere else for academic credit, either by ourselves or another person.

We understand what plagiarism implies and declare that this report embodies our own ideas, words, phrases, arguments, graphics, figures, results, and organisation except where reference is explicitly made to another work.

I understand further that any unethical academic behaviour, which includes plagiarism, is seen in a serious light by MNNIT Allahabad and is punishable by disciplinary action as stipulated by the institute's rules and regulations.

(Signatures)

Date – May 18, 2026

Names:

Praveen Rengarajan (20244512),

Vikrant Kumar Singh (20244174),

Suyash Shukla (20246149),

Pratyaksh Tiwari (20246114),

Himanshu Yadav (20246068),

Sourabh Choyal (20246143)

Place – Prayagraj

ACKNOWLEDGEMENTS

We wish to acknowledge with great appreciation towards the numerous, timely and valuable advice of our esteemed mentors: Dhruv Chandhok (20226051), Aryan Vishwakarma (20223057), Alok Gupta (20236019), Krishna Raj Agrawal (20236069) and Vedang Tripathi (20236224) sirs without whom we would have been unable to perform the project to this level.

We also acknowledge each other's continuous efforts as team to go forward with our ideas and the process of implementing them, innovating and trying new things as a team we have learnt a lot together.

We also sincerely thank our family and friends for their support towards compiling this project report.

Date – May 18, 2026

Names:

Praveen Rengarajan (20244512),

Vikrant Kumar Singh (20244174),

Suyash Shukla (20246149),

Pratyaksh Tiwari (20246114),

Himanshu Yadav (20246068),

Sourabh Choyal (20246143)

ABSTRACT

This project aims to create a quadruped robotic dog that accurately imitates the smooth locomotion of a dog in a natural manner. The project combines concepts from robotics, embedded systems, mechanical design and programming to develop a programmable robotic platform capable of stable and coordinated movement.

The structure of the robotic dog has been intensively modelled and simulated using SolidWorks and has been fabricated with high quality PLA to ensure the strength of the parts and its lightweight construction. The use of 3D printing also allows easier assembly, maintenance and future modifications while keeping the structure lightweight and durable.

It is equipped with 12 of the versatile and heavy-duty MG996 servo motors and is operated using an Arduino UNO allowing it to perform realistic walking and navigation after code implementation. The servo motors provide precise joint control and help the robotic dog achieve smooth and coordinated movement.

This robotic dog's movement has been implemented using the concepts of forward and inverse kinematics utilising the movement patterns from the set of rotational and translational matrices that have been implemented through code. These matrices themselves were calculated from the kinematic diagram detailing servo motor motion patterns. This allows the dog to move with a natural gait and translate these matrices into smooth, calculated movement without any jerks in the 3D plane. Path planning has been done to further calculate the positions of the end effector, with which we can make the robotic dog move according to our instructions. The implementation of these concepts improves movement accuracy and synchronization between all four legs.

Before implementing the code on the hardware, simulation of the robotic dog to give us the potential vision was performed using python utilising the same rotational and translation matrices. This helped in analysing the movement patterns and reducing possible hardware errors before practical implementation.

This robot dog can freely move forwards, backwards and sideways through the code implemented in Arduino IDE platform. The integration of simulation and hardware implementation helped improve the stability and smoothness of movement.

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION

- 1.1 General Remarks
- 1.2 Project Overview
- 1.3 Applications
- 1.4 Objectives of the Project

CHAPTER 2: TECHNICAL DETAILS

- 2.1 System Block Diagram
- 2.2 Hardware Setup & Requirements
- 2.3 Software Architecture
- 2.4 Coordinate System and Offsets

CHAPTER 3: MODELING AND KINEMATICS

- 3.1 Manipulator Design
- 3.2 Forward Kinematics and Transformation Matrices
- 3.3 Inverse Kinematics Approach
- 3.4 Simulation and Control
- 3.5 Path planning

CHAPTER 4 : PROBLEMS FACED AND FUTURE SCOPE

- 4.1 Problems faced and solution to them
- 4.2 Future scope of the project

CHAPTER 5: CONCLUSIONS

- 4.1 Conclusions

4.2 Future Scope

REFERENCES

LIST OF FIGURES

Figure 2.1: CAD Assembly of the Robotic Dog Chassis.....	12
Figure 2.2: Final Assembly of the Robotic Dog.....	12
Figure 2.3: Code Snippet for Path Planning.....	13
Figure 3.1: Free Body/Manipulator Diagram of a Single Robotic Leg.....	14
Figure 3.2: Rotational and Translational Matrices Obtained from Fig 3.1.....	15
Figure 3.3: Final Matrices.....	15
Figure 3.4: Python 3D Simulation.....	17

LIST OF TABLES

Table 2.1: Hardware Components List.....	11
--	----

CHAPTER 1

INTRODUCTION

1.1. General Remarks

The Quadruped Robotic Dog often fascinates most people when they first see it but the complex engineering behind it, and its quite many applications in our modern world make it an attractive project to attempt for aspiring robotics enthusiasts. Quadruped robotic systems have gained significant attention in recent years due to their ability to imitate the movement and adaptability of four-legged animals in complex environments where conventional wheeled robots may fail. This project often fascinates most people when they first see it but the complex engineering behind it, and its quite many applications in our modern world make it an attractive project to attempt for aspiring robotics enthusiasts. With recent advancements in embedded systems, actuator technology, 3D printing and robotic control algorithms, it has become possible to develop lightweight and efficient robotic platforms capable of realistic locomotion and stable movement. Such systems are finding applications in robotics research, surveillance, industrial inspection and search-and-rescue operations, making quadruped robotics an important and rapidly developing area in modern engineering and automation.

Legged robotics is a rapidly advancing field of mechatronics that seeks to emulate the locomotion of biological organisms. Unlike wheeled or tracked vehicles, legged robots are highly adaptable to uneven terrain, stairs, and unstructured environments. Developing such a system requires a deep understanding of spatial mathematics, specifically forward and inverse kinematics, to coordinate multiple degrees of freedom simultaneously.

1.2. Project Overview

This project encompasses the design, simulation, and hardware realization of a four-legged robotic dog. The quadruped features four identical legs (Front Right,

Front Left, Back Right, Back Left), each engineered with three degrees of freedom driven by high-torque servo motors.

The core of the system relies on an inverse kinematics solver that determines the necessary joint angles to place the end effectors (the feet) at a specific XYZ coordinate. Python scripts provide a virtual 3D visualization to test joint movements and bounds safely, while Arduino microcontrollers translate these calculated angles into PWM (Pulse Width Modulation) signals to drive the physical robot.

1.3. Applications

Quadrupedal robots have vast potential applications. The fundamental principles explored in this project lay the groundwork for:

- Search and rescue operations in hazardous or collapsed structures.
- Inspection of industrial environments (e.g., pipelines, uneven manufacturing floors).
- Educational platforms for teaching advanced kinematics and control theory.

1.4 Objectives of the project

- To study and implement the principles of quadruped robotics and legged movement mechanisms.
- To develop a real-time control system for coordinating multiple servo motors.
- To simulate and test locomotion patterns such as walking and gait generation.
- To gain practical knowledge in robotics, embedded systems, motion planning, and control algorithms.

CHAPTER 2

TECHNICAL DETAILS

2.1. Hardware Setup & Requirements

The physical chassis is controlled via an Arduino-compatible microcontroller. Actuation is achieved using twelve servo motors, allocating three servos per leg to replicate the hip, knee, and ankle joints.

Table 2.1: Hardware Components List

Components

S No.	Name	Description	Quantity
1.	Arduino UNO	Microcontroller board that controls servo motors and coordinates the robotic dog's leg movements.	1
2.	MG996R Servo Motors	joint movement for hip and knee, 3 motors/leg	12
3.	Chassis Material (Acrylic/PLA)	Frame to mount all components.	1
4.	SMPS (12V, 10A)	Power source for servos and controller.	1
5.	Buck Converter (LM2596)	Voltage regulator to supply 6.8V to servos.	1

8.	Misc. Hardware (screws, brackets, servo horns, Jumper Wires & Connectors etc.)	Mechanical assembly of robot legs and body & Electrical connections between components.	-
----	--	--	---

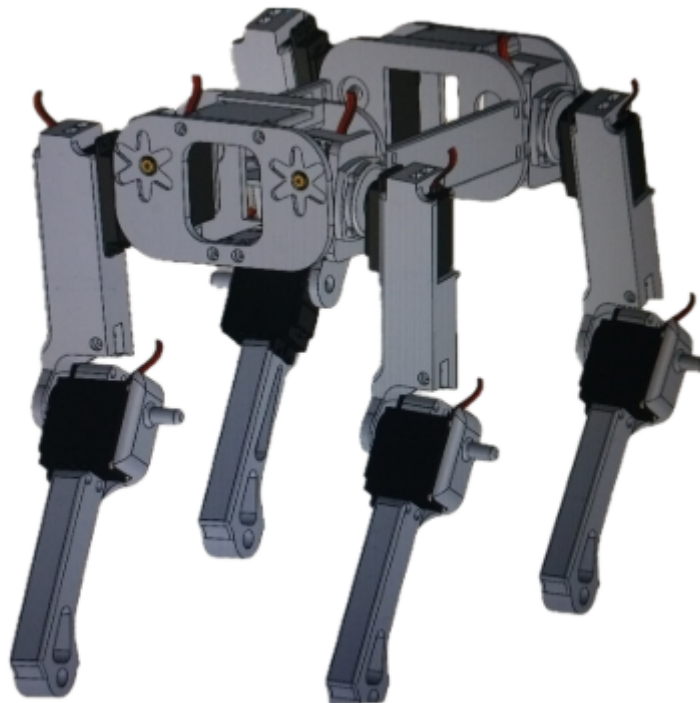


Figure 2.1: CAD Assembly of the Robotic Dog Chassis

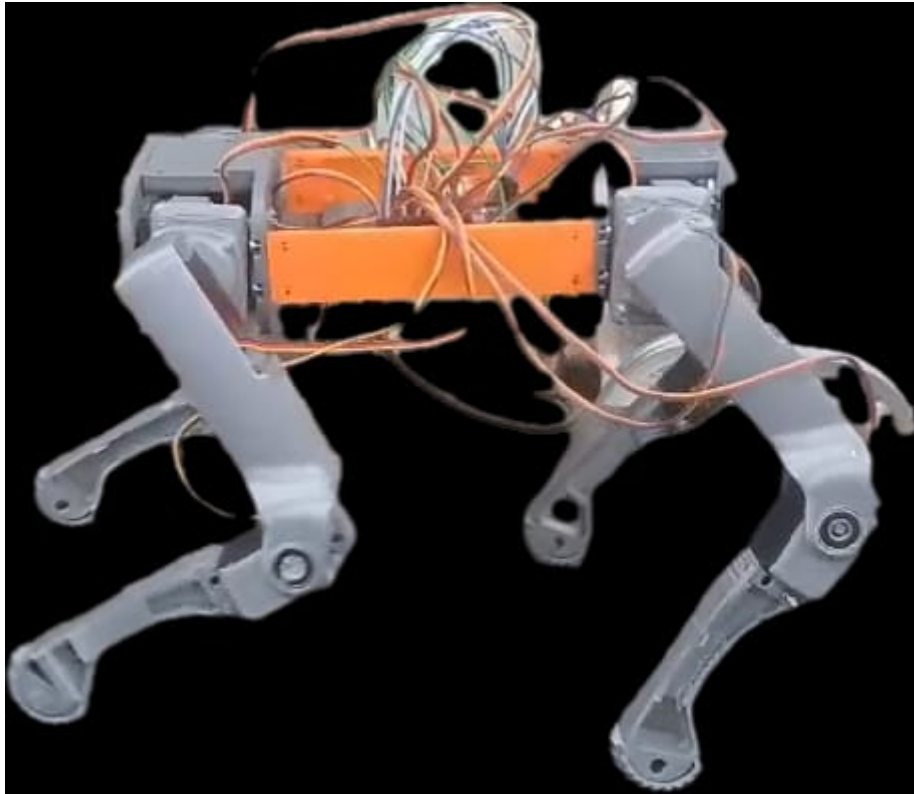


Figure 2.2: Final Assembly of the Robotic Dog

2.2. Software Architecture

The software is divided into two distinct environments:

1. **Simulation Environment (Python 3.x):** Utilizes `numpy` for matrix calculations and `matplotlib` for 3D visualization. The scripts (`4 Legs Simulation.py`, `Left Leg Simulation.py`, etc.) allow for visual debugging of the kinematics before deploying to hardware.
2. **Hardware Firmware (Arduino C++):** Individual `.ino` files process target coordinates inputted via Serial Monitor and command the servos on arduino digital pins

```

void cubicMove(float aL[3], float bL[3],
               float aR[3], float bR[3],
               float t, int steps, int delay_ms) {

    for (int j = 0; j <= steps; j++) {

        float i = (t / steps) * j;

        th1 = aL[0] + (3 * (bL[0] - aL[0]) / (t * t)) * i * i
                - (2 * (bL[0] - aL[0]) / (t * t * t)) * i * i * i;

        th2 = aL[1] + (3 * (bL[1] - aL[1]) / (t * t)) * i * i
                - (2 * (bL[1] - aL[1]) / (t * t * t)) * i * i * i;

        th3 = aL[2] + (3 * (bL[2] - aL[2]) / (t * t)) * i * i
                - (2 * (bL[2] - aL[2]) / (t * t * t)) * i * i * i;

        ag1 = aR[0] + (3 * (bR[0] - aR[0]) / (t * t)) * i * i
                - (2 * (bR[0] - aR[0]) / (t * t * t)) * i * i * i;

        ag2 = aR[1] + (3 * (bR[1] - aR[1]) / (t * t)) * i * i
                - (2 * (bR[1] - aR[1]) / (t * t * t)) * i * i * i;

        ag3 = aR[2] + (3 * (bR[2] - aR[2]) / (t * t)) * i * i
                - (2 * (bR[2] - aR[2]) / (t * t * t)) * i * i * i;
    }
}

```

Fig 2.3 Code snippet gait control

2.5 Power Management System

The voltage requirements for each servo motor was 6.8V for providing the maximum torque of its capability.

We used 12V Switched mode power supply which is an electronic circuit that converts power using high frequency power regulators.

We then converted from 12V to 6.8V by using a BUCK converter and then powered it to all servos.

CHAPTER 3

MODELING AND KINEMATICS

3.1. Manipulator Design

Each leg operates as an open-loop kinematic chain. The geometry of the physical robot dictates the lengths of the linkages:

- Hip: $a_1 = 4.5\text{cm}$
- Thigh: $a_2 = 6.0\text{cm}$
- Calf: $a_3 = 5.4\text{cm}$

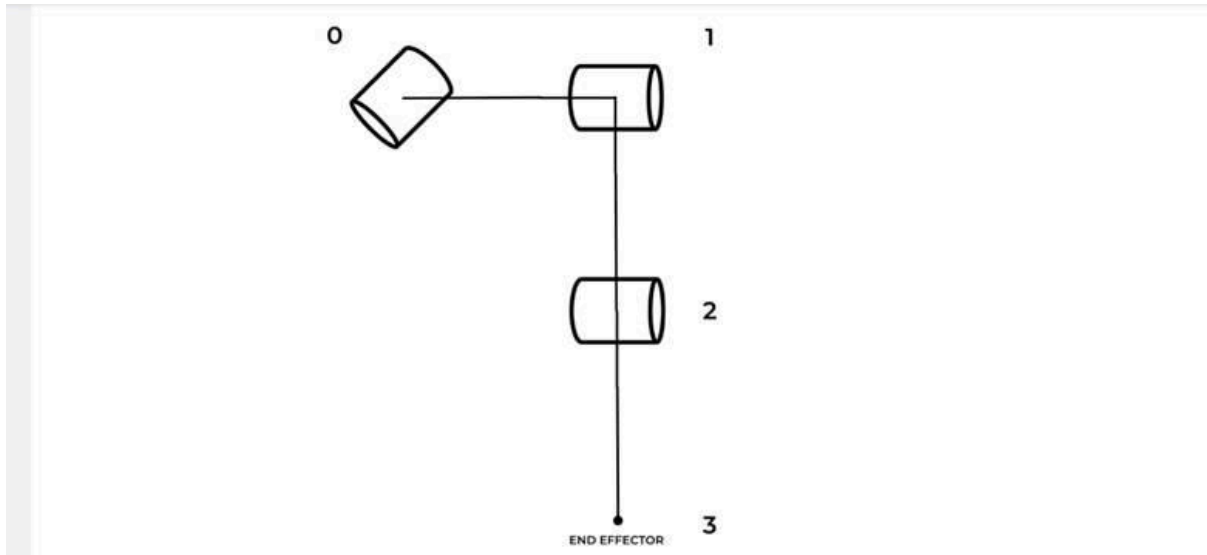


Figure 3.1: Free body/manipulator diagram of a single robotic leg.

3.2. Forward Kinematics and Transformation Matrices

Forward kinematics involves calculating the final position of the end effector given specific joint angles (T_1 , T_2 , T_3). This is achieved by multiplying the Homogeneous Transformation Matrices of each joint.

The rotation matrices corresponding to the joints are defined as follows:

$$\begin{aligned}
 R_{0 \rightarrow 1} &= \begin{bmatrix} -\sin(T_1) & 0 & \cos(T_1) \\ \cos(T_1) & 0 & \sin(T_1) \\ 0 & 1 & 0 \end{bmatrix} & D_{0 \rightarrow 1} &= \begin{bmatrix} a_1 \cos(T_1) \\ a_1 \sin(T_1) \\ 0 \end{bmatrix} \\
 R_{1 \rightarrow 2} &= \begin{bmatrix} \cos(T_2) & -\sin(T_2) & 0 \\ \sin(T_2) & \cos(T_2) & 0 \\ 0 & 0 & 1 \end{bmatrix} & D_{1 \rightarrow 2} &= \begin{bmatrix} a_2 \cos(T_1) \\ a_2 \sin(T_2) \\ 0 \end{bmatrix} \\
 R_{2 \rightarrow 3} &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & -1 & 0 \end{bmatrix} & D_{2 \rightarrow 3} &= \begin{bmatrix} a_3 \cos(T_3) \\ a_3 \sin(T_3) \\ 0 \end{bmatrix}
 \end{aligned}$$

Figure 3.2: Rotational and Translational Matrices obtained from Fig 3.1

$$\begin{aligned}
 H_1 &= \begin{bmatrix} -\sin(\theta_1) & 0 & \cos(\theta_1) & a_1 \cos(\theta_1) \\ \cos(\theta_1) & 0 & \sin(\theta_1) & a_1 \sin(\theta_1) \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\
 H_2 &= \begin{bmatrix} \cos(\theta_2) & \sin(\theta_2) & 0 & a_2 \cos(\theta_2) \\ -\sin(\theta_2) & \cos(\theta_2) & 0 & -a_2 \sin(\theta_2) \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\
 H_3 &= \begin{bmatrix} \cos(\theta_3) & \sin(\theta_3) & 0 & a_3 \cos(\theta_3) \\ -\sin(\theta_3) & \cos(\theta_3) & 0 & -a_3 \sin(\theta_3) \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}
 \end{aligned}$$

Figure 3.3: Final Matrices.

3.3. Inverse Kinematics Approach

While forward kinematics is useful for simulation, moving the physical robot requires inverse kinematics. The user provides a target coordinate (x, y, z), and the system must calculate the required joint angles.

A python code was written based on these equations to verify the forward kinematics of the dog

```

import math
a1, a2, a3 = 2.7, 11.3, 13
def clamp(x, lo=-1.0, hi=1.0):
    return max(lo, min(hi, x))
def ik_xyz(x, y, z, a1=a1, a2=a2, a3=a3, degrees=
True):
    sols = []
    r = math.hypot(x, y)
    if r < abs(a1) - 1e-9:
        return sols
    phi = math.atan2(y, x)
    cos_alpha = clamp(a1 / r)
    alpha = math.acos(cos_alpha)
    for alpha_val in [alpha, -alpha]:
        t1 = phi - alpha_val
        Xp = r * math.sin(alpha_val)
        Zp = z
        dist2 = Xp**2 + Zp**2
        D = clamp((dist2 - a2**2 - a3**2) / (2 *
a2 * a3))
        if abs(D) > 1:
            continue

```

```

        for sgn in (+1.0, -1.0):
            sin_t3 = sgn * math.sqrt(max(0.0, 1.0
- D**2))
            t3 = math.atan2(sin_t3, D)
            num = a3 * math.sin(t3)
            den = a2 + a3 * math.cos(t3)
            t2 = math.atan2(Zp, Xp) - math.atan2(
num, den)
            t2_servo = -t2
            t3_servo = -t3
            if degrees:
                sols.append((
                    math.degrees(t1),
                    math.degrees(t2_servo),
                    math.degrees(t3_servo)
                ))
            else:
                sols.append((t1, t2_servo,
t3_servo))
        return sols
if __name__ == "__main__":
    x_in = float(input("Enter x: "))
    y_in = float(input("Enter y: "))
    z_in = float(input("Enter z: "))
    solutions = ik_xyz(x_in, y_in, z_in)
    if not solutions:
        print("No IK solution.")
    else:
        print("First IK solution (servo angles):"
)
        print(tuple(round(v, 4) for v in
solutions[0]))

```

Because the left and right legs are mirrored in their physical assembly, the acceptable angle bounds differ to prevent self-collision and hardware damage.

3.4. Simulation and Control

We created a software program called Single Leg Simulation.py which enabled us to perform tests to verify the leg motion of a single leg of the robotic dog without needing a simulation for the full 4 legs. Using interactive UI sliders we were able to manipulate the joint variables for T1, T2, and T3 and see the motion of the leg as it was defined by the mathematical model in real-time. The simulation also provided a way to view the leg dynamically in the X-Y Plane, X-Z Plane, and Y-Z Plane so that we could see the leg move more easily.

```

import numpy as np
import matplotlib.pyplot as plt
from matplotlib.widgets import Slider

def c(theta): return np.cos(np.radians(theta))
def s(theta): return np.sin(np.radians(theta))
a1, a2, a3 = 2.7, 11.3, 13
theta1_init, theta2_init, theta3_init = 0,0,0
def compute_positions(theta1, theta2, theta3):
    H1 = np.array([
        [-s(theta1), 0, c(theta1), a1*c(theta1)
    ]],
        [ c(theta1), 0, s(theta1), a1*s(theta1)
    ]],
        [0, 1, 0, 0],
        [0, 0, 0, 1]
    ])
    H2 = np.array([
        [ c(theta2), s(theta2), 0, a2*c(theta2)],
        [-s(theta2), c(theta2), 0, -a2*s(theta2)
    ]],
        [0, 0, 1, 0],
        [0, 0, 0, 1]
    ])
    H3 = np.array([
        [ c(theta3), s(theta3), 0, a3*c(theta3)
    ]],
        [-s(theta3), c(theta3), 0, -a3*s(theta3)
    ]],
        [0, 0, 1, 0],
        [0, 0, 0, 1]
    ])

```

```

    T1 = H1
    T2 = H1 @ H2
    T3 = T2 @ H3
    O0 = np.array([0, 0, 0])
    O1 = T1[:3, 3]
    O2 = T2[:3, 3]
    O3 = T3[:3, 3]
    return np.array([O0, O1, O2, O3])
fig = plt.figure(figsize=(8, 7))
ax = fig.add_subplot(111, projection='3d')
plt.subplots_adjust(bottom=0.25)
positions = compute_positions(theta1_init,
theta2_init, theta3_init)
(line,) = ax.plot(positions[:, 0], positions[:, 1],
positions[:, 2],
                    '-o', lw=3, color='royalblue')
eff_text = ax.text2D(0.05, 0.95,
                    f"End-Effector: X={positions
[-1,0]:.2f}, Y={positions[-1,1]:.2f}, Z={
positions[-1,2]:.2f}",
                    transform=ax.transAxes,
                    fontsize=12, color='darkred')
reach = abs(a1) + abs(a2) + abs(a3)
ax.set_xlim([-reach, reach])
ax.set_ylim([-reach, reach])
ax.set_zlim([0, reach])
ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')
axcolor = 'lightgoldenrodyellow'

```

```

ax_theta1 = plt.axes([0.15, 0.15, 0.65, 0.03],
facecolor=axcolor)
ax_theta2 = plt.axes([0.15, 0.10, 0.65, 0.03],
facecolor=axcolor)
ax_theta3 = plt.axes([0.15, 0.05, 0.65, 0.03],
facecolor=axcolor)
s_theta1 = Slider(ax_theta1, 'Theta1', -90,90,
valinit=theta1_init)
s_theta2 = Slider(ax_theta2, 'Theta2', -90,90,
valinit=theta2_init)
s_theta3 = Slider(ax_theta3, 'Theta3', -90, 90,
valinit=theta3_init)
def update(val):
    t1 = s_theta1.val
    t2 = s_theta2.val
    t3 = s_theta3.val
    pos = compute_positions(t1, t2, t3)
    line.set_data(pos[:, 0], pos[:, 1])
    line.set_3d_properties(pos[:, 2])
    eff_text.set_text(f"End-Effector: X={pos[-1,0]
:.2f}, Y={pos[-1,1]:.2f}, Z={pos[-1,2]:.2f}")
    fig.canvas.draw_idle()
s_theta1.on_changed(update)
s_theta2.on_changed(update)
s_theta3.on_changed(update)
plt.show()

```

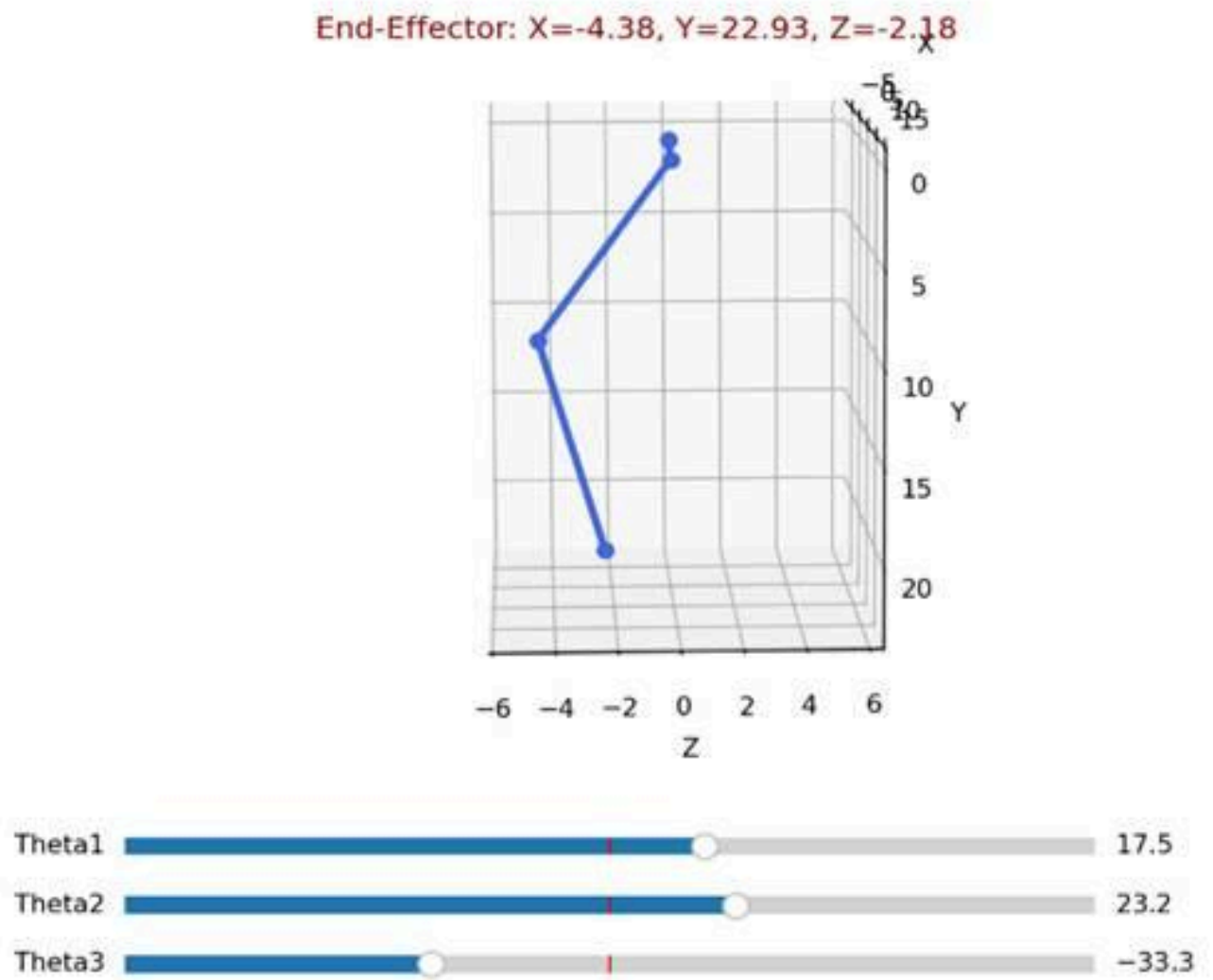


Figure 3.4: Python 3D simulation

3.5. Path Planning

The path planning of the quadruped dog was done using square gait method in which the end effector was allowed to move in the square trajectory.

The equation of the motion of end effector between the two consecutive corners were derived in the following manner:

First we start with a cubic polynomial then following constraints are applied:

Assumed Trajectory Equation

$$\theta(t) = at^3 + bt^2 + ct + d$$

Boundary Conditions

$$\theta(0) = \theta_i, \quad \theta(t_f) = \theta_f, \quad \theta'(0) = 0, \quad \theta'(t_f) = 0$$

Step 1: Apply Initial Position Condition

$$\theta(0) = a(0)^3 + b(0)^2 + c(0) + d$$

$$d = \theta_i$$

Step 2: Differentiate to Obtain Velocity Equation

$$\theta'(t) = 3at^2 + 2bt + c$$

Apply initial velocity condition:

$$\theta'(0) = 3a(0)^2 + 2b(0) + c$$

$$c = 0$$

Step 3: Apply Final Position Condition

$$\theta(t_f) = at_f^3 + bt_f^2 + ct_f + d$$

Substitute $c = 0$ and $d = \theta_i$:

$$\theta(t_f) = at_f^3 + bt_f^2 + \theta_i$$

$$at_f^3 + bt_f^2 = \theta_f - \theta_i$$

Step 4: Apply Final Velocity Condition

$$\theta'(t_f) = 3at_f^2 + 2bt_f + c$$

Substitute $c = 0$:

$$3at_f^2 + 2bt_f = 0$$

$$2bt_f = -3at_f^2$$

$$b = -\frac{3at_f}{2}$$

Step 5: Substitute into Previous Equation

$$at_f^3 + bt_f^2 = \theta_f - \theta_i$$

Substitute value of b :

$$at_f^3 - \left(\frac{3at_f}{2}\right)t_f^2 = \theta_f - \theta_i$$

$$at_f^3 - \frac{3}{2}at_f^3 = \theta_f - \theta_i$$

$$at_f^3 \left(1 - \frac{3}{2}\right) = \theta_f - \theta_i$$

$$-\frac{1}{2}at_f^3 = \theta_f - \theta_i$$

$$a = -\frac{2(\theta_f - \theta_i)}{t_f^3}$$

Step 6: Find b

$$b = -\frac{3at_f}{2}$$

Substitute value of a :

$$b = -\frac{3t_f}{2} \left[-\frac{2(\theta_f - \theta_i)}{t_f^3} \right]$$

$$b = \frac{3(\theta_f - \theta_i)}{t_f^2}$$

Final Cubic Polynomial Trajectory

$$\theta(t) = \theta_i + \frac{3(\theta_f - \theta_i)}{t_f^2}t^2 - \frac{2(\theta_f - \theta_i)}{t_f^3}t^3$$

Further a simulation was created which allowed the user to enter the four coordinates i.e. the four corners of the square in which the end effector was intended to move and it would show the trajectory of the end effector. Using precise analysis of the dog the final angles were calculated

```
import numpy as np
import matplotlib.pyplot as plt
def c(theta): return np.cos(np.radians(theta))
def s(theta): return np.sin(np.radians(theta))
a1, a2, a3 = 2.7, 11.3, 13
def compute_position (theta1, theta2, theta3):
    H1 = np.array([
        [-s(theta1), 0, c(theta1), a1*c(theta1)],
        [ c(theta1), 0, s(theta1), a1*s(theta1)],
        [0, 1, 0, 0],
        [0, 0, 0, 1]
    ])
    H2 = np.array([
        [ c(theta2), s(theta2), 0, a2*c(theta2)],
        [-s(theta2), c(theta2), 0, -a2*s(theta2)],
        [0, 0, 1, 0],
        [0, 0, 0, 1]
    ])
    H3 = np.array([
        [ c(theta3), s(theta3), 0, a3*c(theta3)],
        [-s(theta3), c(theta3), 0, -a3*s(theta3)],
        [0, 0, 1, 0],
        [0, 0, 0, 1]
    ])
    T1 = H1
    T2 = H1 @ H2
    T3 = T2 @ H3
    O0 = np.array([0, 0, 0])
    O1 = T1[:3, 3]
    O2 = T2[:3, 3]
    O3 = T3[:3, 3]
    return np.array([O0, O1, O2, O3])
```

```

def cubic_pat (a, b, t, steps=100):
    l1, l2, l3 = [], [], []
    for j in range(steps + 1):
        i = (t / steps) * j
        theta1 = a[0] + (3*(b[0]-a[0]) / t**2) * i**2
        - (2*(b[0]-a[0]) / t**3) * i**3
        theta2 = a[1] + (3*(b[1]-a[1]) / t**2) * i**2
        - (2*(b[1]-a[1]) / t**3) * i**3
        theta3 = a[2] + (3*(b[2]-a[2]) / t**2) * i**2
        - (2*(b[2]-a[2]) / t**3) * i**3
        l1.append(theta1)
        l2.append(theta2)
        l3.append(theta3)
    return l1, l2, l3

print("Enter 4 points (x y z) ")
pointsz=[]
for i in range(4):
    coords = input(f"Point {i+1}: ").split()
    x, y, z = map(float, coords)
    points.append((x, y, z))

print("\nEnter 4 theta positions (θ1 θ2 θ3 θ4) ")
theta1 = list(map(float, input("Theta 1: ").split()))
theta2 = list(map(float, input("Theta 2: ").split()))
theta3 = list(map(float, input("Theta 3: ").split()))
theta4 = list(map(float, input("Theta 4: ").split()))
t12 = float(input("Time for θ1 → θ2: "))
t23 = float(input("Time for θ2 → θ3: "))
t34 = float(input("Time for θ3 → θ4: "))
t41 = float(input("Time for θ4 → θ1: "))
p12 = cubic_pat (theta1, theta2, t12)
p23 = cubic_pat (theta2, theta3, t23)
p34 = cubic_pat (theta3, theta4, t34)
p41 = cubic_pat (theta4, theta1, t41)
theta1_lis = p12[0] + p23[0] + p34[0] + p41[0]
theta2_lis = p12[1] + p23[1] + p34[1] + p41[1]
theta3_lis = p12[2] + p23[2] + p34[2] + p41[2]
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot (111, projection = '3d')
ax.set_title(
    "3-Link Robotic Leg Simulation with Input Point ")

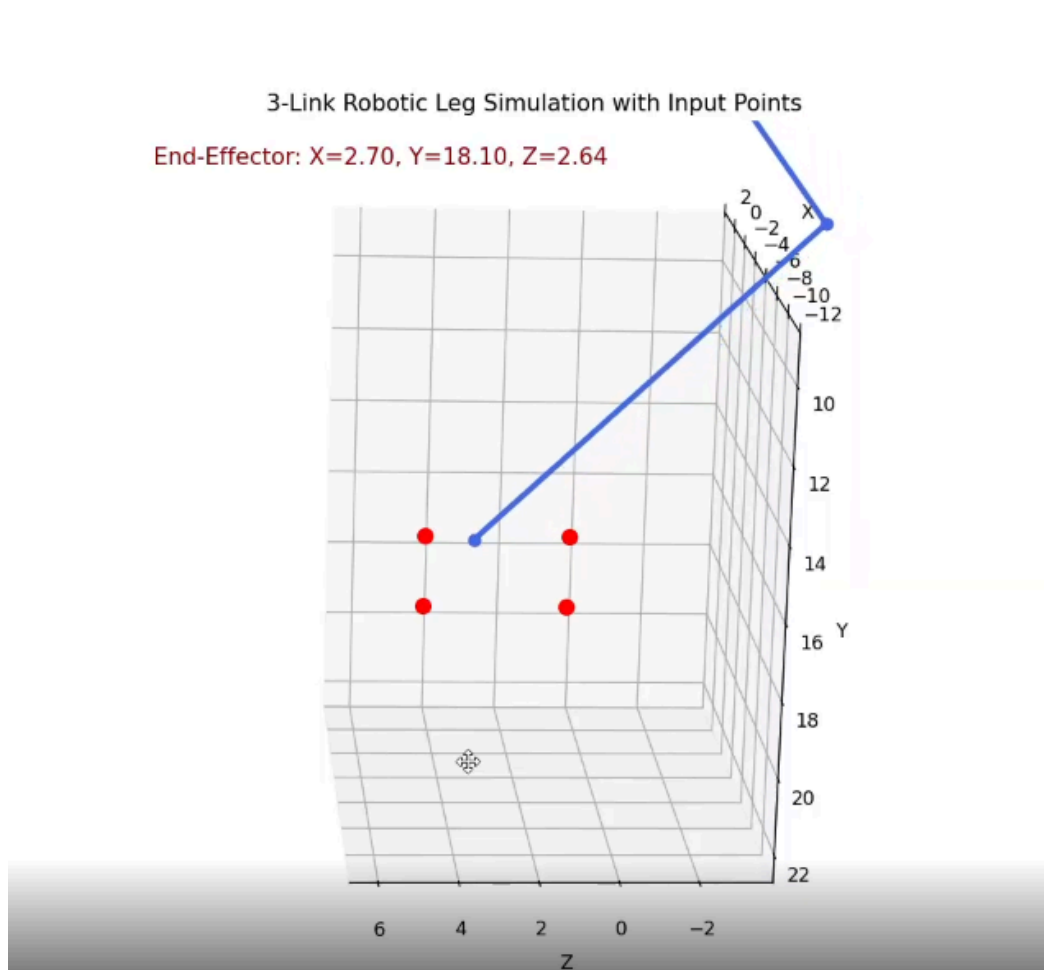
```

```

for x, y, z in points:
    ax.scatter(x, y, z, s=60, c='red')
reach = a1 + a2 + a3
ax.set_xlim([-reach, reach])
ax.set_ylim([-reach, reach])
ax.set_zlim([-reach/2, reach])
ax.set_xlabel("X")
ax.set_ylabel("Y")
ax.set_zlabel("Z")
pos1= compute_position (theta1_lis [0], theta2_lis [0]
], theta3_lis [0])      t      t
(line,) = ax.plot(pos[:, 0], pos[:, 1], pos[:, 2],
'-o', lw=3, c='royalblue')
eff_text = ax.text2D(0.05, 0.95, "", transform=ax.
transAxes, fontsize=12, color='darkred')
plt.ion()
plt.show()
try:
    while True:
        for k in range(len(theta1_lis)):
            if not plt.figure_exists (fig.number):
                raise KeyboardInterrupt
            pos = compute_position (theta1_lis [k],
theta2_lis [k], theta3_lis [k])      t
t      linestyle_data(pos[:, 0], pos[:, 1])
            line.set_3d_properties pos[:, 2])
            eff_text.set_text(f"End-Effector: X {pos[
-1,0]:.2f}, Y={pos[-1,1]:.2f}, Z={pos[-1,2]:.2f}")
            plt.pause(0.02)
except KeyboardInterrupt :
    plt.close()
    print("Simulation close    )
        d."

```

this code could be run to obtain the following simulation



angles used for this simulation were:

	<i>THETA A</i>	<i>THETA B</i>	<i>THETA C</i>	<i>COORDINATES</i>	<i>TIME</i>
CORNER 1	0	37.4804	-69.4127	(2.7,20.0)	1
CORNER 2	0	45.9798	-84.6664	(2.7,18,0)	1
CORNER 3	0	31.6837	-81.523	(2.7,18,4)	1
CORNER 4	0	24.3135	-66.0404	(2.7,20,4)	1

CHAPTER 4

PROBLEMS FACED AND FUTURE SCOPE

4.1 Problems we tackled

1. Servo Motor Synchronization

Problem:

Controlling multiple servo motors simultaneously caused timing mismatches between the legs, resulting in unstable walking and improper gait coordination.

Solution:

We assembled the servo motors by setting their zero angle points at a particular position so that we can get to know in which direction we have to move. Other than this we also provided offsets to the servo for solving the angles mismatch problem.

2. Power Supply Fluctuations

Problem:

The servo motors required high current during operation, leading to voltage drops and unstable system performance.

Solution:

A separate switched regulated power supply with high current output converted by BUCK converter was used for the servo motors, and proper grounding techniques were implemented to ensure stable operation. The arduino was grounded by from the circuit but powered from the laptop to ensure minimum resets.

3. Gait Coordination

Problem:

Generating a smooth walking pattern for four legs simultaneously was challenging during the initial development stage.

Solution:

We chose to implement the square gait trajectory due to its easy implementation and we mirrored the movements of one pair of diagonal legs from other.

4.2 Future Scope

The developed quadruped robotic dog provides a strong foundation for future advancements in the field of robotics and intelligent autonomous systems. The following improvements and extensions can be implemented in future work:

- Integration of computer vision and camera-based object tracking for autonomous navigation and target following.
- Implementation of artificial intelligence and machine learning algorithms to enable adaptive movement and intelligent decision-making.
- Addition of obstacle detection and avoidance using ultrasonic sensors, LiDAR, or depth cameras.
- Development of advanced gait generation techniques for smoother and faster locomotion on uneven terrain.
- Integration of wireless communication technologies such as Wi-Fi or Bluetooth for remote monitoring and control.
- Improvement in balance and stability using feedback sensors such as IMU (Inertial Measurement Unit) and gyroscope modules.

Chapter 5: Design And 3D Printing

The 3D computer-aided design (CAD) of the quadruped robot was developed within the SolidWorks environment. The structural architecture and kinematic layout of this robot are explicitly inspired by the open-source SpotMicro quadruped platform, utilizing a compact, four-legged configuration driven by independent robotic actuators to mimic biological quadrupedal locomotion.

To maintain a clean geometric hierarchy and a computational workflow, a modular bottom-up assembly strategy was implemented. The sequential assembly process is detailed below:

Phase 1: Assembling the Main Chassis (Central Body)

The central body acts as the main base of the robot and houses the internal electronics and brackets.

1. Start a New File: Opened a new SolidWorks Assembly workspace (.sldasm).
2. Insert the Base Plate: Imported the bottom plate of the chassis. Clicking the green checkmark automatically locked it (**Fixed**) to the center of the screen so it cannot move around.
3. Attach the Walls: Attached the left, right, front, and back wall panels to the base plate using Coincident Mates (to touch the flat faces together) and Concentric Mates (to align the screw holes).
4. Mount Internal Brackets: Placed the internal motor housing brackets inside the chassis cavity using Concentric Mates so that the steering motor shafts point outwards toward the corners.

Phase 2: Assembling the Legs

Leg 1 (Front-Left Leg):

1. The Hip Joint: Inserted the first hip bracket and aligned its circular holes with the front-left corner holes of the chassis using a Concentric Mate. Used a Width Mate to center it perfectly.
2. The Upper Leg (Thigh): Connected the upper leg link to the hip bracket using a Concentric Mate so it can swing forward and backward.
3. The Lower Leg (Shin): Connected the lower leg piece to the bottom of the upper leg piece using a Concentric Mate to create a functional knee pivot.

Leg 2 (Front-Right Leg):

1. Attached the second hip bracket to the front-right corner of the chassis using **Concentric** and **Width** mates.
2. Connected the right upper leg link to this hip joint.
3. Attached the right lower leg link to form the right knee joint.

Leg 3 (Rear-Left Leg):

1. Aligned the third hip bracket with the rear-left corner of the chassis.
2. Mated the rear-left upper leg link to the hip bracket.
3. Connected the lower leg piece to complete the rear-left leg assembly.

Leg 4 (Rear-Right Leg):

1. Mounted the final hip bracket onto the remaining rear-right corner of the chassis.
2. Connected the last upper leg link to the rear-right hip joint.
3. Attached the final lower leg piece to finish the fourth leg.

Phase 3: Final Assembly of Bot

Step 1: Opened a New File Started a brand-new SolidWorks Assembly file (.sldasm).

Step 2: Inserted the Main Chassis Assembly Imported the pre-built Main Chassis Sub-Assembly first and clicked the green checkmark to set it as Fixed (f) so it cannot move.

Step 3: Added All Four Legs Brought all four pre-assembled leg parts into this new assembly workspace.

Step 4: Assembled Using Concentric Mates Used the Concentric Mate to align the circular hinge holes of all four legs with the corner mounting points on the main chassis.

3D Printing

- 1: Imported Parts into Ultimaker Cura Opened the 3D design files (.STL) for each robot part inside the Cura slicing software.
- 2: Configured Print Settings for PLA Selected PLA (Polylactic Acid) as the printing material and set up the slicing parameters (such as infill density, and printing temperatures).
- 3: Generated the G-code Clicked the "Slice" button in Cura to convert the 3D models into a G-code file, which contains the toolpath instructions for the printer.
- 4: Transferred the G-code to the 3D Printer Saved the generated G-code onto an SD card (or USB drive) and inserted it directly into the 3D printer.
- 5: Printed the Parts Started the print job on the machine to physically fabricate every part layer-by-layer using the PLA filament.

CHAPTER 6

CONCLUSIONS

6.1. Conclusions

The objective of this project was to successfully design and control a four-legged robotic dog using custom inverse kinematics algorithms. The project successfully bridged mathematical theory with physical application. By deriving accurate transformation matrices and integrating them into a Python simulation environment, the kinematics model was validated.

The implementation of the Arduino firmware proved that the calculated angles could be successfully mapped to real-world servo actuations. The static positional control achieved meets the primary goals of the project, establishing a reliable base frame for future robotics research.

6.2. Future Directions

While static positioning has been achieved, the field of legged locomotion offers vast avenues for expansion. Future improvements for this robotic dog platform include:

- **Dynamic Gait Generation:** Implementing continuous trajectory planning (like trot, creep, and gallop gaits) instead of static point-to-point movements.

- **Sensor Integration:** Incorporating an IMU (Inertial Measurement Unit) to allow the robot to self-balance on inclined or uneven surfaces.
- **Wireless Control:** Transitioning from Serial communication to Wi-Fi or Bluetooth (e.g., using an ESP32) for untethered movement and remote operation.

REFERENCES

[1] J. J. Craig, *Introduction to Robotics: Mechanics and Control*, 3rd ed., Pearson Prentice Hall, 2005.

[2] Arduino Documentation, *Servo Library Reference*. Available: <https://www.arduino.cc/reference/en/libraries/servo/> (Last Accessed: [Insert Date]).

[3] Python Software Foundation, *NumPy and Matplotlib Documentation*. Available: <https://numpy.org/doc/>

[End of Report Draft]